

Lab 2-3

Submission And Assignment Instructions

Submit your `Lab2-3_lastname_firstname.pdf` file on canvas with your answers clearly marked (✅) and code commented. Skeleton code and/or comments are provided throughout to help you.

- ? Questions you must answer or tasks are marked with a " ? "
- ✅ Answers you should give are marked with "✅ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ? YOUR CODE HERE`
 - ★ Bonus questions are marked with a "★ "
 - 🍓 Bonus answers are marked with a "🍓 "

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Regularization Calculations

In this section, we will be performing calculations with and without regularizers to understand how regularization affects weights, loss, and the overall network. The calculations will be performed on ordinary least squares regression networks used in Section 1 of Lab 1 if you need to refer back.

Please round to **two decimal places**.

The formula for an OLS regression will be represented as $\hat{y} = w_1x_1 + w_2x_2$

Important Note

In Lab 1, we defined MSE but forgot to divide by the number of terms (N) since it is a mean. For the purposes of that question, that constant factor is not important. However, make sure whenever you are doing a MEAN, you divide by N!

1.1 Calculate SSE

? Given three datapoints:

$(x_1 = 1, x_2 = 2, y = 4), (x_1 = 2, x_2 = 0, y = 3), (x_1 = 0, x_2 = 1, y = 2)$. In Lab 1, we defined error functions for general OLS. Today we will be using a sum of squares error (SSE) function expressed as $SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$. MSE differs from SSE in that MSE divides by the number of terms (N), but they both produce a square error. Given the SSE of the OLS with the given datapoints, find w_1 and w_2 .

Hint: Find the partial derivatives of w_1 and w_2 (like in Lab 1) and then solve the system of equations to find w_1 and w_2

You may do this math on paper and insert a picture of your work here.

In [1]: #Answer:

```
from IPython.display import Image, display
display(Image("q1.1.jpg"))
```

1.1) Calculate SSE

3 data points $\Rightarrow (1, 0, 4) \quad (2, 0, 3) \quad (0, 1, 2)$ OLS given by: $\hat{y} = w_1 x_1 + w_2 x_2$

$$SSE = \sum_{i=1}^n (y_i - \hat{y})^2 = \sum_{i=1}^n (y_i - (w_1 x_{1i} + w_2 x_{2i}))^2$$

$$\frac{\partial SSE}{\partial w_1} = \sum_{i=1}^n -2(y_i - (w_1 x_{1i} + w_2 x_{2i}))x_{1i}$$

$$\frac{\partial SSE}{\partial w_2} = \sum_{i=1}^n -2(y_i - (w_1 x_{1i} + w_2 x_{2i}))x_{2i}$$

$$\sum_{i=1}^n -2x_{1i}(y_i - (w_1 x_{1i} + w_2 x_{2i})) = 0$$

$$\sum_{i=1}^n -2x_{2i}(y_i - (w_1 x_{1i} + w_2 x_{2i})) = 0$$

$$\sum_{i=1}^n -2x_{1i}y_i + w_1 \sum_{i=1}^n 2x_{1i}^2 + w_2 \sum_{i=1}^n 2x_{1i}x_{2i} = 0$$

$$\sum_{i=1}^n -2x_{2i}y_i + 2w_1 \sum_{i=1}^n x_{1i}x_{2i} + 2w_2 \sum_{i=1}^n x_{2i}^2 = 0$$

compute solving data points

$$\begin{aligned} (-1)(4) + (-2)(3) + (0)(2) + w_1 [(1)^2 + (2)^2 + (0)^2] + w_2 [(1)(2) + (2)(0) + (0)(1)] &= 0 \\ -4 - 6 + w_1 (5) + w_2 (2) &= 0 \\ -10 + 5w_1 + 2w_2 &= 0 \rightarrow 5w_1 + 2w_2 = 10 \end{aligned}$$

$$\begin{aligned} (-4)(2) + (-3)(0) + (-2)(1) + w_1 [(1)(2) + (2)(0) + (0)(1)] + w_2 [(0)^2 + (0)^2 + (1)^2] &= 0 \\ -8 - 2 + w_1 (2) + w_2 (1) &= 0 \\ -10 + 2w_1 + 5w_2 &= 0 \rightarrow 2w_1 + 5w_2 = 10 \end{aligned}$$

Solve the system

$$\begin{aligned} 5w_1 + 2w_2 &= 10 \quad (\times 2) \\ 2w_1 + 5w_2 &= 10 \quad (\times 5) \end{aligned}$$

$$\begin{aligned} 10w_1 + 4w_2 &= 20 \\ 10w_1 + 25w_2 &= 50 \end{aligned} \Rightarrow (10w_1 + 25w_2) - (10w_1 + 4w_2) = 50 - 20$$

$$21w_2 = 30$$

$$w_2 = \frac{30}{21} = \frac{10}{7}$$

$$w_1 = \frac{10}{7}$$

$$\begin{aligned} 5w_1 + 2\left(\frac{10}{7}\right) &= 10 \\ 5w_1 + \left(\frac{20}{7}\right) &= 10 \\ 5w_1 &= 10 - \frac{20}{7} \\ 5w_1 &= \frac{70}{7} - \frac{20}{7} \\ w_1 &= \frac{50}{5 \cdot 7} = \frac{50}{35} = \frac{10}{7} \end{aligned}$$

1.2 Calculation of Lasso and Ridge Regression

? For $\lambda = 1$, calculate the SSE with no regularizer, SSE with lasso regularization (L1), and SSE with ridge regression (L2). You should be reporting 3 answers.

Hint: Remember no regularizer is just SSE, lasso is $\lambda(w_1 + w_2)$, and ridge is $\lambda(w_1^2 + w_2^2)$.

In [2]: #Answer:

```
from IPython.display import Image, display
display(Image("q1.2.jpg"))
```

1.2 - Calculation of Lasso & Ridge Regularization

$$\begin{aligned} \textcircled{1} \text{ SSE} &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \left[4 - \left(\frac{10}{7}(-1) + \frac{10}{7}(2)\right)\right]^2 + \left[3 - \left(\frac{10}{7}(2) + \frac{10}{7}(0)\right)\right]^2 + \left[2 - \left(\frac{10}{7}(0) + \frac{10}{7}(-1)\right)\right]^2 \\ &= \left(4 - \frac{20}{7}\right)^2 + \left(3 - \frac{20}{7}\right)^2 + \left(2 - \frac{10}{7}\right)^2 \\ &= \frac{3}{7} \approx 0.43 \end{aligned}$$

$$\begin{aligned} \textcircled{2} \text{ SSE with } L_1 &= \frac{3}{7} + \lambda \sum_{j=1}^n |w_j| \quad \text{LASSO} \\ &= \frac{3}{7} + 1 \left(\left|\frac{10}{7}\right| + \left|\frac{10}{7}\right|\right) \\ &= \frac{3}{7} + \frac{20}{7} \\ &= \frac{23}{7} \approx 3.29 \end{aligned}$$

$$\begin{aligned} \textcircled{3} \text{ SSE with } L_2 &= \frac{3}{7} + \lambda \sum_{j=1}^n w_j^2 \quad \text{RIDGE} \\ &= \frac{3}{7} + 1 \left(\left(\frac{10}{7}\right)^2 + \left(\frac{10}{7}\right)^2\right) \\ &= \frac{221}{49} \approx 4.51 \end{aligned}$$

1.3 Understanding Regularizer Penalty

? What would happen to the loss as λ changes? What does this mean for the weights and overall loss?

✓ Answer: λ is what we often call the shrinkage parameter. Therefore, if λ increases, the coefficients are much more penalized, which could result in a loss of important information and lead to underfitting. If λ decreases, the coefficients are less penalized, which could lead to overfitting. The regularization effect on the loss would decrease.

1.4 Understanding Ridge and Lasso

? Is the loss for the L1 or L2 regularizer greater? In general, how does L1 vs L2 regularization affect the weights?

✓ Answer: The loss for the Ridge Regression Regularizer (L2) was greater ($4.51 > 3.29$). In general, Lasso Regularization (L1) performs feature selection by setting some coefficients to 0. This means it can eliminate certain features it deems not important by driving their coefficients exactly to zero.

On the other hand, Ridge Regularization does not perform feature selection but instead shrinks all coefficients toward zero, reducing or increasing their influence without removing features entirely.

Section 2: k-NN Visualizations with Toy Data

2.0 Explaining kNN

? Briefly explain kNN inference using any arbitrary distance metric.

✓ Answer: For example, you have a test data point and you want to know which y label it is associated with. You compute its distance (for example, Euclidean distance) between itself and all training points, then rank those distances in ascending order. You then decide what you want to set your hyperparameter k to (for example, $k=1, 2$, or 3). If $k=3$, you look at the closest three neighbors and assign the y label based on the majority class among those three neighbors for classification, or the mean of their values for regression.

2.1: Visualizing Decision Boundaries

Run the code provided below to visualize the toy dataset and decision boundaries for several different distance metrics and values of k .

```
In [3]: ## provided toy dataset code
# load necessary packages
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from matplotlib.colors import ListedColormap

# Generate toy dataset, set seed to ensure reproducibility
np.random.seed(42)
n_points_per_class = 20

# create 3 overlapping classes of data
class1 = np.random.randn(n_points_per_class, 2) + [2, 1]
class2 = np.random.randn(n_points_per_class, 2) + [1, -2]
class3 = np.random.randn(n_points_per_class, 2) + [0, 0]

# combine these points into X and y datasets
X = np.vstack((class1, class2, class3))
y = np.array([0] * n_points_per_class + [1] * n_points_per_class + [2] * n_points_per_class)

# Plotting function to show decision boundaries
def plot_decision_boundaries(X, y, metric, ks):
    n_cols = len(ks)
    fig, axes = plt.subplots(1, n_cols, figsize=(n_cols * 6, 6))

    cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
    cmap_bold = ['red', 'green', 'blue']
    markers = ['o', 's', 'D']

    for ax, k in zip(axes, ks):
        clf = KNeighborsClassifier(n_neighbors=k, metric=metric)
        clf.fit(X, y)

        h = .02 # step size in the mesh (just for plotting decision boundaries)
        # prep plotting parameters for groups and decision boundaries
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
```

```

xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

## plotting the outputs
ax.contourf(xx, yy, Z, cmap=cmap_light)
for i, color, marker in zip(range(3), cmap_bold, markers):
    sns.scatterplot(ax=ax, x=X[y == i, 0], y=X[y == i, 1], marker=marker,
                    edgecolor='k', s=50, label=f'Class {i}')

ax.set_xlim(xx.min(), xx.max())
ax.set_ylim(yy.min(), yy.max())
# make sure to label the plot with the metric and value of k
ax.set_title(f"K-NN (k={k}, metric='{metric}')" )
ax.legend()

plt.tight_layout()
plt.show()

```

```

# Choose distance metrics and vector of values of k to display
metrics = ['euclidean', 'manhattan']
ks = [1, 3, 10]

```

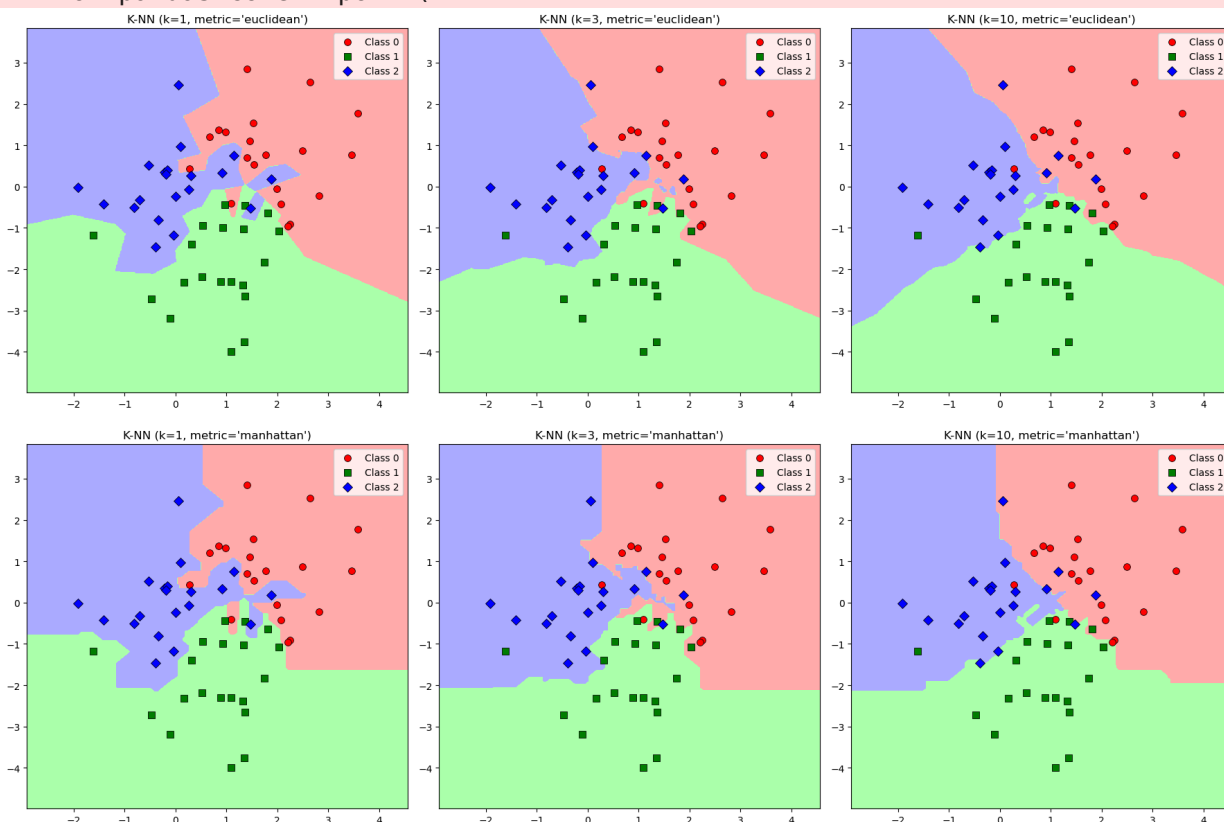
```

# call function to plot decision boundaries for different metrics and values of k
for metric in metrics:
    plot_decision_boundaries(X, y, metric, ks)

```

/Users/dianadayekh/anaconda3/lib/python3.11/site-packages/pandas/core/arrays/masked.py:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).

```
from pandas.core import *
```



? 2.1.1 Briefly describe similarities and differences between the different values of k in the plots above. How do the decision boundaries change with different values of k ? Explaining this in terms of underfitting and overfitting. Based on your knowledge of how the data were generated in the code above, are particular values of k more or less accurate classifiers?

✓ Answer: For the $k=1$ plot, we are searching for the single nearest neighbor and assigning that y label to the point. In this case, the point itself is used, meaning there is a correct label for each point, as we can see in the plots. The decision boundaries are very clearly defined, with each class in its own region, and there is 100% accuracy. However, this indicates that we are overfitting to the data. As k increases, for example $k=3$ or $k=10$, we start to see some mislabeled points. Some green points appear in the blue region, and some blue points appear in the red region, and so on. Therefore, accuracy decreases; however, we are overfitting the data less and may start to underfit by $k=10$. Based on how the data were generated, $k=3$ should be the best choice because it balances overfitting and underfitting.

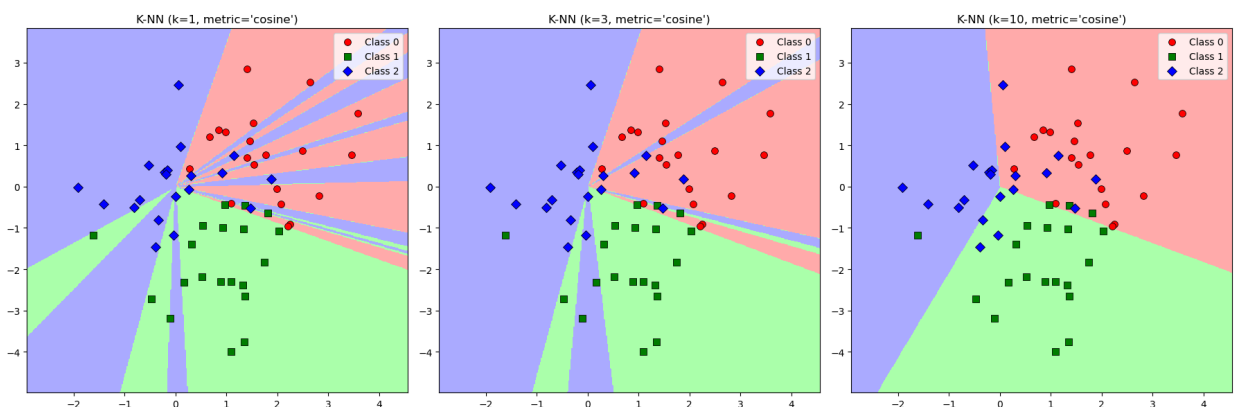
? 2.1.2 Based on the way these data were generated in the code above, are there particular classes of data that should be more or less difficult to separate from the other(s)? Do you see that reflected in the plots and decision boundaries?

✓ Answer: The blue and red classes are much harder to separate because their data points are more spread out and may overlap. This results in a more complex decision boundary being required to distinguish between them.

? 2.1.3 Using the same data and the same values of k , run the k -NN algorithm again using a cosine distance metric. *Hint: You should just have to call the `plot_decision_boundaries` function written above with slightly different parameters.*

```
In [4]: # Choose distance metrics and vector of values of k to display
metrics = ['cosine']
ks = [1, 3, 10]

# call function to plot decision boundaries for different metrics and values of k
for metric in metrics:
    plot_decision_boundaries(X, y, metric, ks)
```



? 2.1.4 Cosine similarity can be thought to measure "directionality" of the data. What kind of errors occur for this data and cosine distance? Based on how the data was generated,

why is this the case?

✓ Answer: Using cosine distance, errors occur when points from different classes lie in the same direction from the origin. Because cosine distance only measures direction and ignores magnitude, points that are far apart but aligned along the same ray can be considered very similar and classified together. This leads to misclassification when different classes overlap in angle. The data was generated as Euclidean clusters around different centers, so class membership depends on distance, not direction, which makes cosine similarity misaligned with the true class structure.

? 2.1.5 What are examples or use cases where cosine similarity should be used?

✓ Answer: Cosine similarity should be used when you are interested in the direction of a data point from the origin rather than its magnitude, for example when the relative pattern of features matters more than their absolute values, such as in text documents where document length may vary but the topic remains the same.

2.2: Splits & Cross-validation - choosing the best value of k

? 2.2.1 Choose an appropriate distance metric (*hint: not cosine similarity*) and add onto the code below to rerun the exercise above with cross-validation and test the vector of ks (`ks`) provided. Report the accuracy for each value of k you test and create a plot of cross-validation accuracy vs. k . Add error bars for standard error of the mean to the cross-validation accuracy plots.

A commented framework is provided below. Code blocks you need to complete are marked with `## ? YOUR CODE HERE` at the top.

```
In [5]: ## you don't need to modify this code block, just read it
## some newly necessary packages for cross-validation and error
from sklearn.model_selection import cross_val_score, KFold
from sklearn.metrics import accuracy_score

# Generate toy dataset
np.random.seed(0)
n_points_per_class = 50

class1 = np.random.randn(n_points_per_class, 2) + [2, 1]
class2 = np.random.randn(n_points_per_class, 2) + [1, -2]
class3 = np.random.randn(n_points_per_class, 2) + [0, 0]

# combine these 3 classes of data together into one dataset and one set of class labels
X = np.vstack((class1, class2, class3))
y = np.array([0] * n_points_per_class + [1] * n_points_per_class + [2] * n_points_per_class)

# vector of k values to test with cross-validation
ks = range(1,31)

# make new plotting function to just deal with 1 value of k (no subplots of multiple k values)
def plot_decision_boundaries_2(X, y, metric, k):
```



```

fig, ax = plt.subplots(figsize=(6, 6))

cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
cmap_bold = ['red', 'green', 'blue']
markers = ['o', 's', 'D']

clf = KNeighborsClassifier(n_neighbors=k, metric=metric)
clf.fit(X, y)

h = .02 # step size in the mesh (just for plotting decision boundaries)
# prep plotting parameters for groups and decision boundaries
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

## plotting the outputs
ax.contourf(xx, yy, Z, cmap=cmap_light)
for i, color, marker in zip(range(3), cmap_bold, markers):
    sns.scatterplot(ax=ax, x=X[y == i, 0], y=X[y == i, 1], marker=marker,
                    edgecolor='k', s=20, label=f'Class {i}')

ax.set_xlim(xx.min(), xx.max())
ax.set_ylim(yy.min(), yy.max())
# make sure to label the plot with the metric and value of k
ax.set_title(f'K-NN (k={k}, metric='{metric}'))
ax.legend()

plt.tight_layout()
plt.show()

```

In [6]:

```

## ? YOUR CODE HERE
def evaluate_knn(X, y, ks, metric):
    # Write a function to fit the K-NN classifier and evaluate accuracy using 5-
    # use random_state = 12 for the cross-validation
    kf = KFold(n_splits=5, shuffle=True, random_state=12)
    all_fold accuracies = [] # will store one list per k (each list has 5 fold)

    # loop through different values of k to run cross-validation on each

    for k in ks:
        fold accuracies = [] # store 5 accuracies (one per fold)

        for train_idx, test_idx in kf.split(X):
            X_train, X_test = X[train_idx], X[test_idx]
            y_train, y_test = y[train_idx], y[test_idx]

            # fit k-NN on training fold
            clf = KNeighborsClassifier(n_neighbors=k, metric=metric)
            clf.fit(X_train, y_train)

            # test on test fold
            y_pred = clf.predict(X_test)
            acc = accuracy_score(y_test, y_pred)
            fold accuracies.append(acc)
            all_fold accuracies.append(fold accuracies)

    # return classification accuracy scores from the cross-validation

```



```

return np.array(all_fold accuracies) # shape: (len(ks), 5)

# choose a distance metric
metric = 'euclidean'
# apply the above function for your chosen distance metric
accuracy_scores = evaluate_knn(X, y, ks, metric)

# calculate mean and std error of accuracy across the 5 folds of cross-validation
mean_acc = accuracy_scores.mean(axis=1)
std_acc = accuracy_scores.std(axis=1, ddof=1)
se_acc = std_acc / np.sqrt(5)

# Plot accuracy with error bars for each value of k
plt.figure(figsize=(6,4))
plt.errorbar(ks, mean_acc, yerr=se_acc, fmt='o-')
plt.xlabel('k (number of neighbors)')
plt.ylabel('Cross-validated accuracy')
plt.title(f'5-fold CV accuracy vs k (metric={metric})')
plt.show()

# find the best value of k and the corresponding accuracy
# make sure to print out the best value of k and corresponding accuracy
best_idx = np.argmax(mean_acc)
best_k = ks[best_idx]
best_acc = mean_acc[best_idx]
print("Best k:", best_k)
print("Best mean CV accuracy:", best_acc)

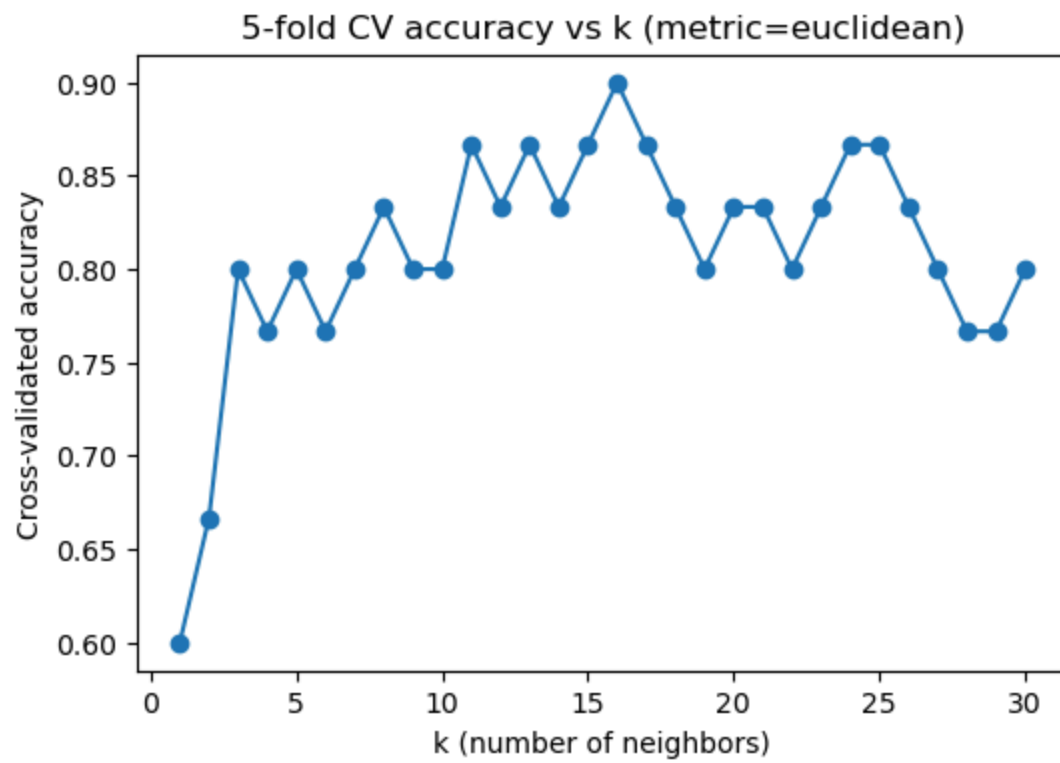
# Plot decision boundaries for the best value of k (not all values of k!)
# the function from above is sufficient for this
plot_decision_boundaries_2(X, y, metric, best_k)
plt.show()

```

```

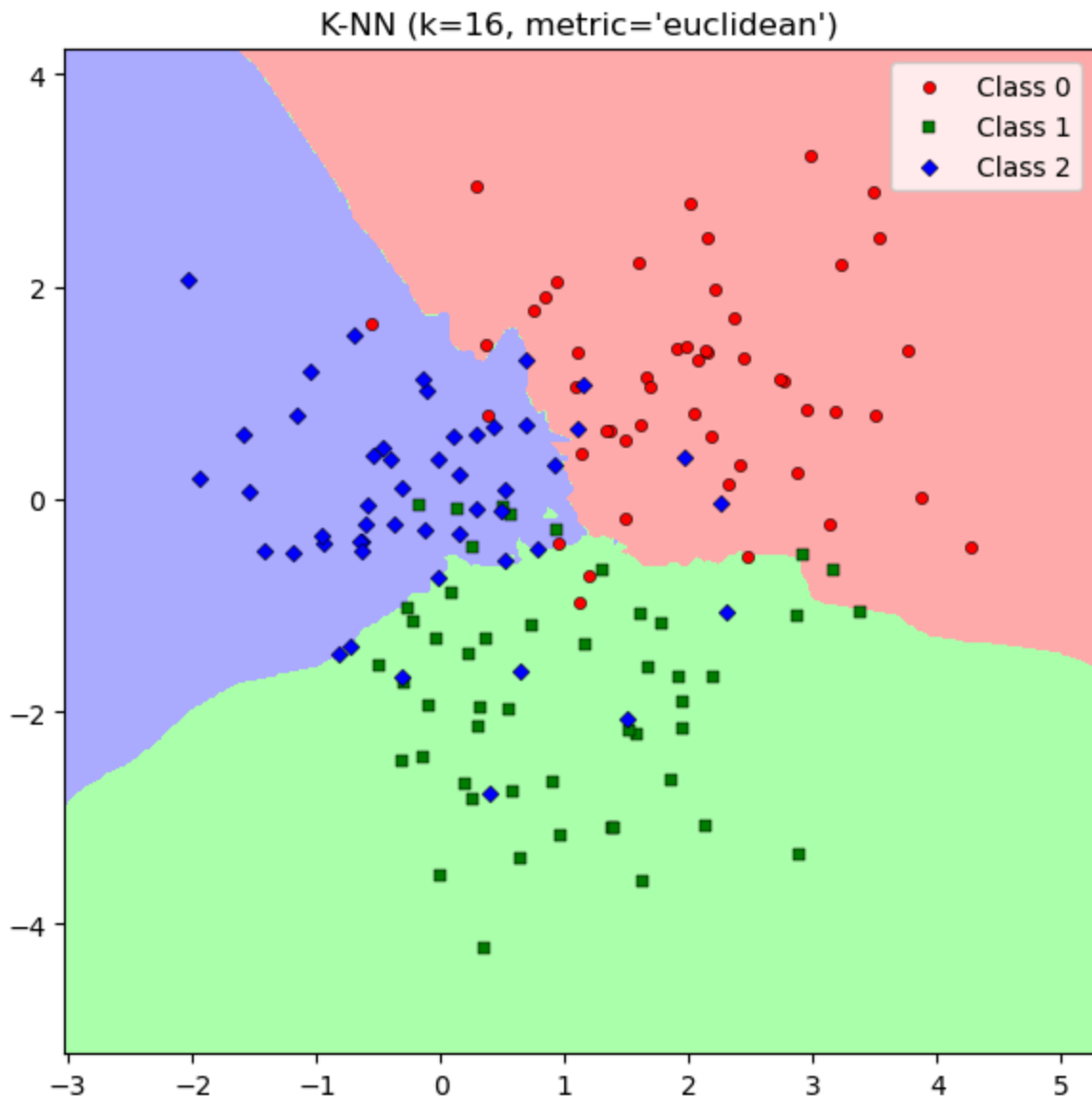
/Users/dianadayekh/anaconda3/lib/python3.11/site-packages/numpy/core/_methods.
py:206: RuntimeWarning: Degrees of freedom <= 0 for slice
  ret = _var(a, axis=axis, dtype=dtype, out=out, ddof=ddof,
/Users/dianadayekh/anaconda3/lib/python3.11/site-packages/numpy/core/_methods.
py:195: RuntimeWarning: invalid value encountered in divide
  ret = um.true_divide(
/Users/dianadayekh/anaconda3/lib/python3.11/site-packages/matplotlib/axes/_bas
e.py:2503: UserWarning: Warning: converting a masked element to nan.
  xys = np.asarray(xys)

```



Best k: 16

Best mean CV accuracy: 0.9



? 2.2.2 What is the best value of k for these data? Based on the plots of average cross-validation accuracy across various values of k , recommend and defend a range of appropriate values of k .

✓ Answer: The best k value is $k = 16$ given that it yielded the highest mean accuracy of 0.9. If we observe the average cross-validation accuracy plot across various values of k , the highest means were achieved for k values ranging from $k = 15$ to $k = 17$.

Section 3: Multiple Linear Regression

3.1: Load the data, prep for modelling

Run the following codeblock to upload the file `data_01_wdbc.data.csv`

```
In [7]: # upload local file "data01_breastcancer.csv"
from google.colab import files
uploaded = files.upload()
```

```

for fn in uploaded.keys():
    print('User uploaded file "{name}" with length {length} bytes'.format(
        name=fn, length=len(uploaded[fn])))

# load file
# be sure to specify the header explicitly for this data, as it isn't included
colnames = ["ID", "diagnosis",
            "radius1", "texture1", "perimeter1", "area1", "smoothness1", "compactness",
            "concave_points1", "symmetry1", "fractal_dimension1",
            "radius2", "texture2", "perimeter2", "area2", "smoothness2", "compactness",
            "concave_points2", "symmetry2", "fractal_dimension2",
            "radius3", "texture3", "perimeter3", "area3", "smoothness3", "compactness",
            "concave_points3", "symmetry3", "fractal_dimension3"]

```

```

-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[7], line 2
      1 # upload local file "data01_breastcancer.csv"
----> 2 from google.colab import files
      3 uploaded = files.upload()
      5 for fn in uploaded.keys():

ModuleNotFoundError: No module named 'google'

```

? 3.1.1 Load the uploaded `.csv` file with breast cancer information as a pandas dataframe, as before. Check that you've loaded the data correctly and specified the header correctly by visualizing the first 5 rows. *You should have 569 observations of 32 features.*

```

In [8]: # Import pandas
# read in the uploaded file as a pandas df, specify the colnames vector above
# note that there is no header in data_01_wdbc.data.csv file
import pandas as pd

colnames = ["ID", "diagnosis",
            "radius1", "texture1", "perimeter1", "area1", "smoothness1", "compactness",
            "concave_points1", "symmetry1", "fractal_dimension1",
            "radius2", "texture2", "perimeter2", "area2", "smoothness2", "compactness",
            "concave_points2", "symmetry2", "fractal_dimension2",
            "radius3", "texture3", "perimeter3", "area3", "smoothness3", "compactness",
            "concave_points3", "symmetry3", "fractal_dimension3"]

## ? YOUR CODE HERE
df = pd.read_csv(
    "data_01_wdbc.data.csv",
    header=None,
    names=colnames
)
# visualize the first 5 rows to check your work
df.head(5)

#csv_name = "data_01_wdbc.data.csv"
#cancer_data = pd.read_csv(csv_name, names=colnames)

```

Out [8]:

	ID	diagnosis	radius1	texture1	perimeter1	area1	smoothness1	compactness1	co
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	

5 rows × 32 columns

? 3.1.2 Split your data into predictors and output variable. You'll need to create a dummy variable of 0s and 1s to code for "benign" or "malignant" samples, respectively. Make sure to drop columns you don't want to use as predictors from the predictor dataframe (e.g. ID probably shouldn't be used as a predictor variable).

```
In [9]: ## ? YOUR CODE HERE
import numpy as np
# Create a dummy variable for output (tumor diagnosis)
y = df["diagnosis"].map({"B": 0, "M": 1})

# quick check that nothing was mapped to NaN
print(y.isna().sum())

## Create separate X dataframe for predictors
X = df.drop(columns=["ID", "diagnosis"])

0
```

? 3.1.3 Split the data randomly into a train and test set; we will compare error rates and metrics on the test data to contrast models. Use `random_state = 42` to make this split reproducible across the class. Call your new predictor dataframes `X_train` and `X_test` for compatibility with the TA scaling code below.

```
In [10]: # load train_test_split for this task
# reproducibly split the data into 80% training and 20% testing
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.2,
    random_state=42
)
```

Standardize and scale your predictor variables before building models.

Hint: Should the entire dataset be included in the scaling calculation?

```
In [11]: # load StandardScaler to scale data
from sklearn.preprocessing import StandardScaler

## ? YOUR CODE HERE
scaler = StandardScaler()
```

```
# fit and transform the training data
X_train_scaled = scaler.fit_transform(X_train)

# apply the same transformation to the test data - use the parameters
X_test_scaled = scaler.transform(X_test)
```

? 3.1.4 Are we standardizing on just entire dataset or just one specific part? If so, why does this occur?

✓ Answer: We standardize using only the training data and the same transformation is applied to the test set so that it is evaluated using statistics derived only from the training data.

Standardizing on the entire dataset would introduce bias because Information from the test set would influence the scaling parameters, leading to an overly optimistic evaluation of the model's performance. Using only the training data ensures a truthful and unbiased assessment on the test set.

3.2: Logistic Regression Model Fitting

? 3.2.1 Fit a logistic regression model using all the available continuous predictors to predict whether the tissue is benign or malignant. Be sure *not* to add regularization - the [documentation](#) makes it clear that L2 regularization is the default choice.

```
In [12]: # load the model I want to use
# Fitting a logistic regression model
from sklearn.linear_model import LogisticRegression

X_train_regression = LogisticRegression(
    penalty=None,
    solver="lbfgs",
    max_iter=1000
)

X_train_regression.fit(X_train_scaled, y_train)
```

```
Out[12]: LogisticRegression
LogisticRegression(max_iter=1000, penalty=None)
```

? 3.2.2 Fit a second model, this time with L2 regularization.

```
In [13]: ## ? YOUR CODE HERE
# use L2 regularization to build a model

X_train_regression2 = LogisticRegression(
    penalty="l2",
    solver="lbfgs",
    max_iter=1000
)

X_train_regression2.fit(X_train_scaled, y_train)
```

Out[13]:

```

▼ LogisticRegression
LogisticRegression(max_iter=1000)

```

3.3: Evaluating the Models

? 3.3.1 Use both models to make predictions on *both* the training and test data. Report the accuracy for each model on both training and test data.

In [14]:

```

# load evaluation metrics
from sklearn.metrics import accuracy_score

## ? YOUR CODE HERE
## Make predictions on the test data, using all 3 models
#MODEL 1: Logistic regression without Regularization + #calculate accuracy for
y_train_predicted = X_train_regression.predict(X_train_scaled)
y_test_predicted = X_train_regression.predict(X_test_scaled)

train_acc_1 = accuracy_score(y_train, y_train_predicted)
test_acc_1 = accuracy_score(y_test, y_test_predicted)

#MODEL 2: Logistic regression with L2 loss
y_train_predicted2 = X_train_regression2.predict(X_train_scaled)
y_test_predicted2 = X_train_regression2.predict(X_test_scaled)

train_acc_2 = accuracy_score(y_train, y_train_predicted2)
test_acc_2 = accuracy_score(y_test, y_test_predicted2)

## display accuracy values
#print("Accuracies of full, and l2 models on test data:\n", accuracies,
#      "\nAccuracies of full, and l2 models on training data:\n", accuracies_t
print("For No Regularization:")
print(f"Training data: {train_acc_1:.3f}")
print(f"Test data: {test_acc_1:.3f}")
print("")
print("For L2-Regularization:")
print(f"Training data: {train_acc_2:.3f}")
print(f"Test data: {test_acc_2:.3f}")

```

```

For No Regularization:
Training data: 1.000
Test data: 0.939

```

```

For L2-Regularization:
Training data: 0.987
Test data: 0.974

```

? 3.3.2 Which model has the highest accuracy on the *training* data? Is this what you expect? Why or why not? (1 sentence).

✓ Answer: The model without regularization achieves higher training accuracy (100%) than the L2-regularized model (98.7%), which is expected because removing regularization allows the model to overfit the training data, while L2 regularization constrains the coefficients and reduces overfitting.

? 3.3.3 Compare and discuss the accuracies on the *test* data. Consider overfitting.

✓ Answer: The L2-regularized model achieves higher accuracy on the test data (97.4%) than the unregularized model (93.9%). This occurs because the unregularized model overfits the training data by learning noise rather than the underlying pattern, which harms its performance on new unseen data (test data). In contrast, L2 regularization constrains the model which reduces overfitting and yields a higher accuracy rate on the test data.

★ Bonus - 3.4: Evaluating Models with ROC Curves

Since this is a logistic regression model that actually outputs probabilities we threshold into classifications, we can sketch out a ROC curve for each model.

★ 3.4.1 Make a ROC curve for each model using the testing data and calculate the AUC for each. For the purposes of this assignment, also make a ROC curve for each model on the training data, not just the testing data. Make sure to label your axes appropriately.

Hint: We've asked for several different ROC curves and their AUC! You could write a function to print a ROC curve and display the AUC, given a model and data (training or test X and y values). Skeleton code is provided below.

```
In [15]: # load necessary evaluation metrics
from sklearn.metrics import roc_auc_score, confusion_matrix, roc_curve, auc

## ? YOUR CODE HERE
## Write function to make a ROC curve and calculate AUC
def plot_my_roc_curve(model, X_data, y_values):
    # write your code here
    # predict probabilities of the positive class (malignant tumor)
    prob_1 = model.predict_proba(X_data)[:, 1]

    # Compute ROC curve and AUC
    fpr, tpr, thresholds = roc_curve(y_values, prob_1)
    roc_auc = auc(fpr, tpr)

    ## plotting
    plt.figure(figsize = (4,4))
    # Display ROC curve and AUC
    plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, zorder=3)

    # add a line for where random chance is on the ROC curve
    plt.plot([0, 1], [0, 1], linestyle="--")
    # change the limits of the x and y axis so you can see the full plot
    plt.xlim(0, 1)
    plt.ylim(0, 1)
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve")
    plt.show()

    return roc_auc

# Apply this function to your test data
```

```

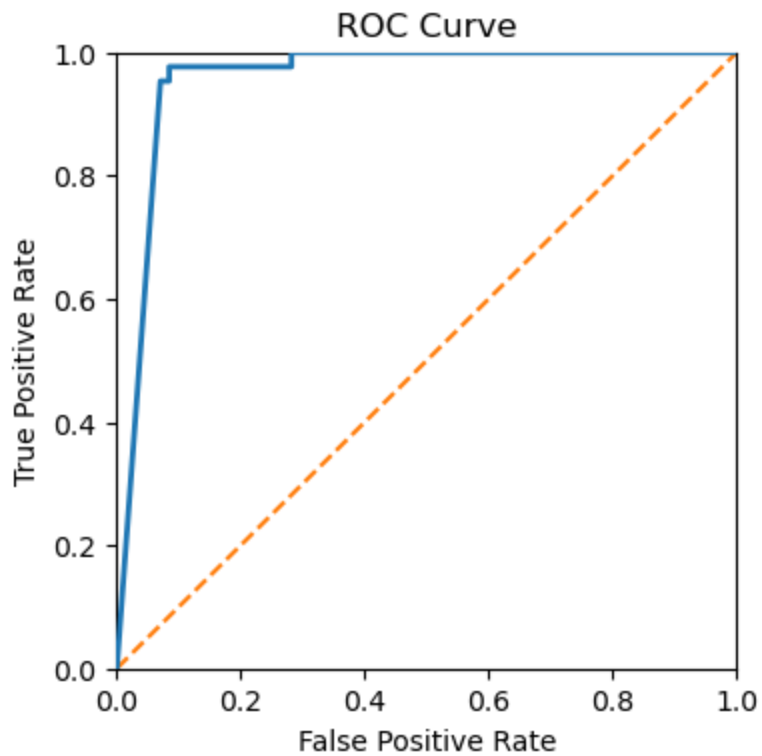
test_auc_no_reg = plot_my_roc_curve(X_train_regression, X_test_scaled, y_test)
print(f"Test AUC no regularization: {test_auc_no_reg:.3f}")

test_auc_l2 = plot_my_roc_curve(X_train_regression2, X_test_scaled, y_test)
print(f"Test AUC with L2: {test_auc_l2:.3f}")

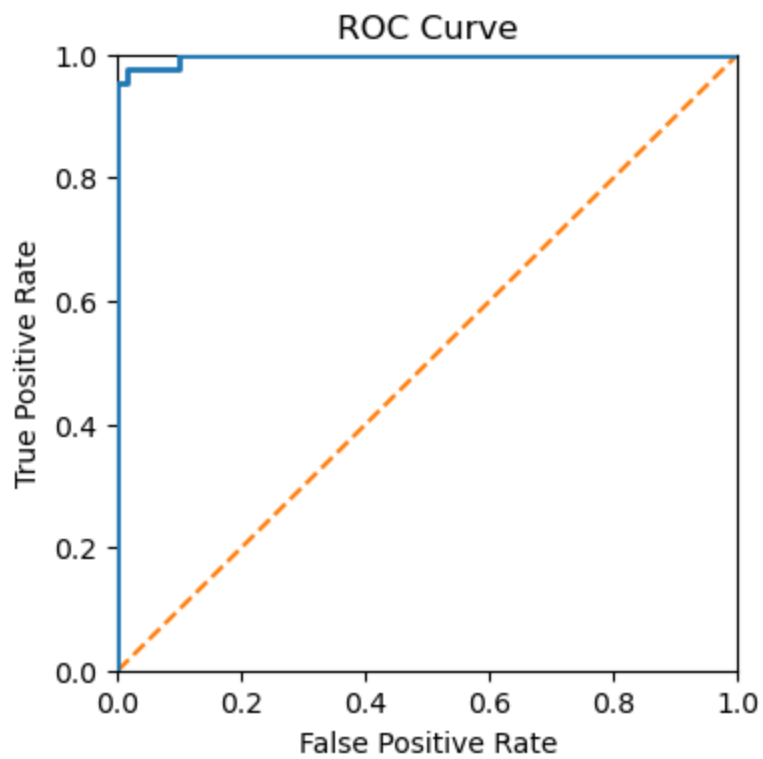
# Apply this function to your training data
train_auc_no_reg = plot_my_roc_curve(X_train_regression, X_train_scaled, y_train)
print(f"Train AUC no regularization: {train_auc_no_reg:.3f}")

train_auc_l2 = plot_my_roc_curve(X_train_regression2, X_train_scaled, y_train)
print(f"Train AUC with L2: {train_auc_l2:.3f}")

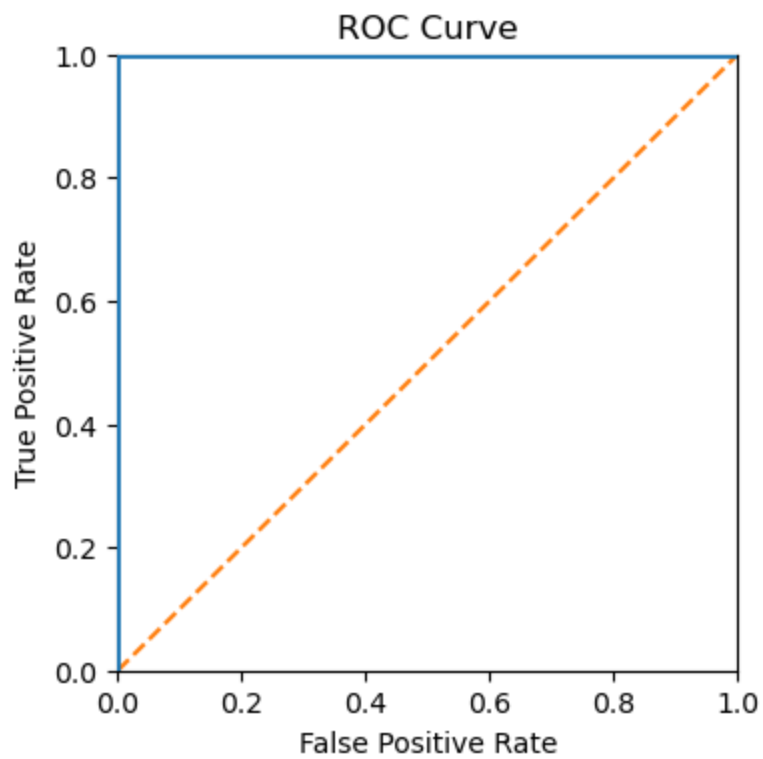
```



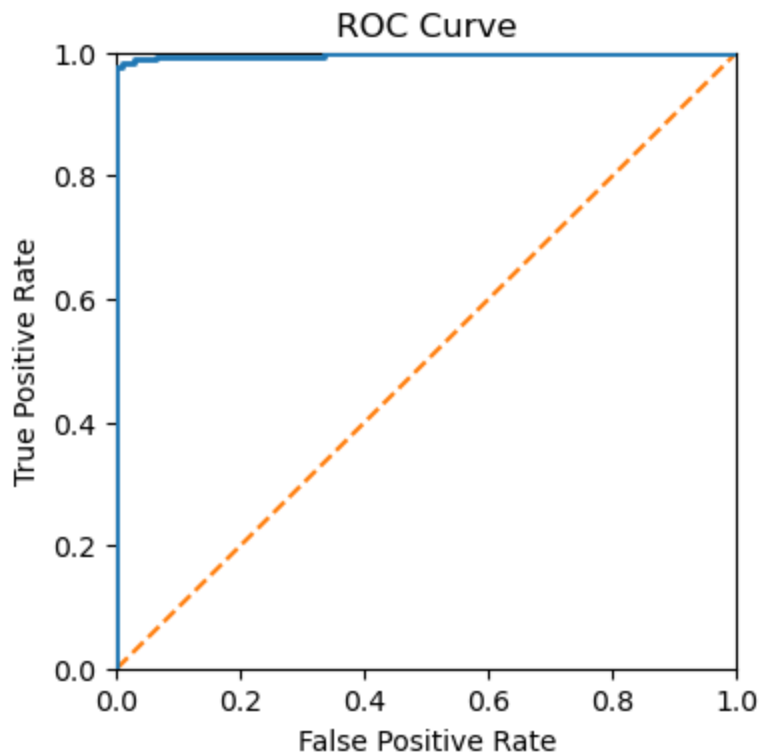
Test AUC no regularization: 0.958



Test AUC with L2: 0.997



Train AUC no regularization: 1.000



Train AUC with L2: 0.997

★ 3.4.2 Briefly compare and contrast your ROC curves and AUC on both models across training and testing data.

🍓 Answer: Across the training set, we can see that Model 1 (with no regularization) performs perfectly, achieving an AUC of 1.00. This indicates that the model achieves a 100% true positive rate while producing no false positives across thresholds, resulting in a perfect area under the ROC curve. This behavior can be explained by the overfitting nature of Model 1, which fits the training data extremely closely. On the other hand, Model 2 with L2 regularization performs slightly worse on the training data, with an AUC of 0.997, because regularization penalizes coefficients and prevents the model from overfitting as much as Model 1.

However, when evaluating performance on the test data, Model 2 (with L2 regularization) yields a higher AUC than Model 1 (0.997 vs. 0.958). This is expected because Model 1 overfits the training data and learns noise rather than the underlying pattern. In contrast, the L2-regularized model is better able to capture the true structure of the data, which allows it to maintain a higher true positive rate while controlling false positives on unseen data.

In []: